

SIKSHA 'O' ANUSANDHAN
(Deemed To Be University) Accredited
(3rd Cycle) by NAAC with A⁺⁺ Grade

Admission Batch: 2023-27

Session:2025-26

Major Project Report of
MACHINE LEARNING CONCEPT -2 (CSE 3968)
On

Fake News Detection Using NLP

Submitted by:

Name	Registration number
1) Avinandan Roy	2341018208
2) Avnish Kumar	2341011148
3) Sourav Malik	2341013081
4) Ujjwal Kumar Singh	2341019489
5) Ankit Kumar	2341011131

Section: 23412J2

Semester:6th

Branch: CSE



Department of Computer Science & Engineering
Institute of Technical Education and Research (ITER), S'O'A
Jagamohan Nagar, Jagamara, Bhubaneswar, Odisha – 751030

(MAY, 2026)

1. Abstract

This project presents a machine learning-based Fake News Detection System that classifies news articles as real or fake using Natural Language Processing (NLP) techniques. It uses supervised learning algorithms such as Decision Tree, Passive Aggressive Classifier, Logistic Regression, and Random Forest for text classification.

The dataset is preprocessed using NLP techniques like tokenization, stopwords removal, and TF-IDF vectorization to convert raw text into numerical features and improve model performance. The system provides quick and reliable predictions, assisting in the automated identification of false information in news content.

This project aims to combat the rapid spread of misinformation online by offering a reliable detection tool, helping readers and platforms verify the authenticity of news articles. It serves to reduce the societal impact of misleading information by promoting more informed media consumption.

2. Table of Contents

Page No	Content
04	Introduction
05	Problem Statement
06	Objectives
07	Literature Review
08	Methodology
09	System Architecture
10 – 11	Implementation
12	Results
13	Discussion
14	Applications
15	Future Scope
16	Conclusion
17	References
18 - 31	Appendix
32	Individual Contributions Table

3. Introduction

- ❖ Natural Language Processing (NLP) and Machine Learning play a vital role in the modern digital era by enabling the automated analysis of large volumes of text.
- ❖ They help in identifying deceptive patterns, linguistic cues, and misinformation from vast amounts of news data that are difficult to detect manually.
- ❖ Fake news detection is a crucial application of these technologies, allowing the rapid classification of news articles as authentic or fabricated.
- ❖ Identifying false information early helps in preventing panic, reducing societal division, and maintaining public trust in digital media.
- ❖ Many internet users struggle to verify the authenticity of online news, which leads to the rapid and harmful spread of misleading information.
- ❖ There is a strong need for an intelligent system that can assist users and platforms in filtering and understanding the credibility of news content.
- ❖ This project uses NLP techniques like TF-IDF alongside classification algorithms such as Logistic Regression, Naïve Bayes, and Random Forest to detect fake news.
- ❖ The system acts as a preliminary content verification tool and supports readers in making informed decisions about the media they consume.
- ❖ It is designed to combat digital misinformation and improve the overall quality of information shared across social media and digital platforms.
- ❖ However, the system is an assistive tool for credibility scoring and should be used alongside critical thinking and professional fact-checking sources.

4. Problem Statement

Readers often struggle to identify fake news accurately based on article content due to the highly persuasive and deceptive nature of fabricated information. This can lead to the rapid spread of misinformation, unwarranted public panic, societal polarization, and a general loss of trust in digital media. Additionally, access to professional, real-time fact-checking sources is often limited or delayed during everyday browsing, making immediate verification difficult.

This project aims to develop a Natural Language Processing (NLP) and machine learning-based system that classifies news articles as real or fake based on text inputs. The system analyzes linguistic features, semantic meaning, and structural patterns, providing a credibility assessment using classification algorithms.

The goal is to assist in the early-stage detection of fabricated news and provide a reliable content-verification tool, helping users make informed decisions about their media consumption while not replacing professional journalistic verification or critical thinking.

5. Objectives

- ❖ To develop a binary classification model that accurately predicts whether a news article is real or fake based on its text content.
- ❖ To apply and compare different Natural Language Processing (NLP) techniques and machine learning algorithms such as Logistic Regression, Naïve Bayes, and Random Forest.
- ❖ To improve the accuracy and reliability of fake news detection using proper text preprocessing (such as TF-IDF vectorization, tokenization) and model evaluation techniques.
- ❖ To assist in the rapid identification of misinformation by providing quick, data-driven credibility assessments.
- ❖ To design a user-friendly system interface for easy input of news text and clear display of classification results.
- ❖ To create a decision-support tool that helps readers and platforms evaluate the authenticity of digital news before consuming or sharing it as fact.

6. Literature Review

- ❖ Several fake news detection systems have been developed using machine learning and Natural Language Processing (NLP) techniques to assist in the early identification of misinformation. These systems typically use large text-based datasets (such as news articles and social media posts) to train classification models that can categorize news as real or fake with reasonable accuracy. Common approaches include Logistic Regression, Naïve Bayes, and Decision Trees.
- ❖ NLP and machine learning techniques play a crucial role in digital media analysis by enabling semantic understanding and pattern recognition from vast amounts of text data. Algorithms such as Random Forest, Support Vector Machines (SVM), and advanced deep learning models have been widely used for text classification due to their ability to handle complex linguistic structures, deceptive writing patterns, and hidden relationships within news content.
- ❖ Despite advancements, existing systems have certain limitations. Many models suffer from issues such as difficulty detecting sarcasm or nuanced language, reliance on biased or outdated datasets, and reduced accuracy when dealing with newly emerging news topics. Additionally, some systems lack user-friendly interfaces for everyday internet users and proper real-time validation, which affects their practical usability.

7. Methodology

7.1 Data Collection

- Kaggle Fake News Dataset

The dataset contains information about various symptoms and their corresponding diseases, which is used to train the prediction model.

7.2 Data Preprocessing

- We clean the data and remove punctuation marks, white spaces, etc
- We tokenize the data using NLP
- We use TF-IDF and BOW to vectorize the tokenized data into numeric form.
- Noise Reduction (Stopword Removal)
- Feature Refinement

These steps improve data quality and enhance model performance.

7.3 Model Development

Algorithms used:

- Decision Tree
- Passive Aggressive Classifier
- Logistic Regression
- Random Forest

The models are trained on the vectorized text dataset to learn the linguistic patterns and relationships distinguishing real news from fake news.

7.4 Model Evaluation

- Accuracy – measures overall correctness
- Precision – measures correctness of positive predictions
- Recall – measures ability to identify actual cases
- Confusion Matrix – provides detailed performance analysis

These metrics are used to compare models and select the best-performing one.

7.5 System Implementation

- User inputs news text or article content through an interface
 - The trained NLP model processes and vectorizes the input text
 - The system outputs the prediction classifying the news as Real or Fake
-

8. System Architecture

The system architecture represents the overall workflow of the fake news detection system, illustrating how input text data is processed to generate credibility predictions.

❖ **Input Layer:**

The system takes user-input news articles or text, which are provided through an interface. This raw text acts as the primary feature for classification.

❖ **Data Processing Layer:**

The input text is preprocessed using Natural Language Processing (NLP) techniques such as text cleaning, tokenization, stopword removal, and TF-IDF vectorization to convert the words into a machine-readable numerical format.

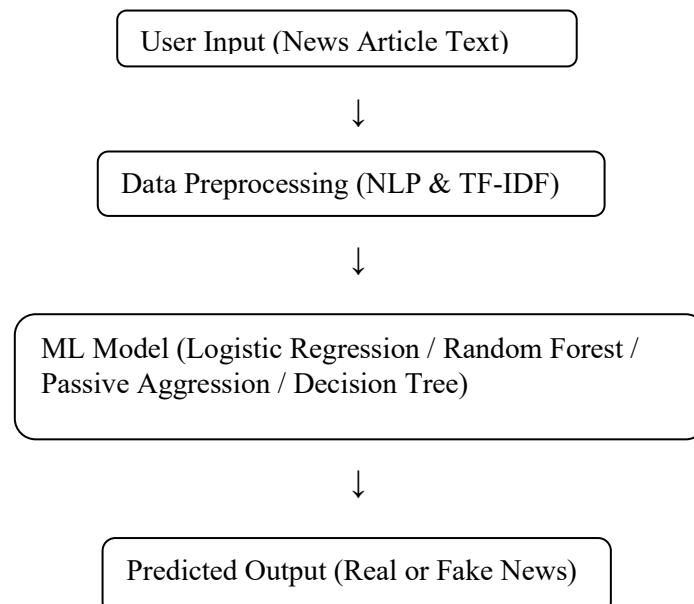
❖ **Model Layer:**

The vectorized data is passed to trained machine learning models such as Logistic Regression, Naïve Bayes, or Random Forest, which analyze the linguistic patterns and semantic features of the text.

❖ **Output Layer:**

The system generates the predicted classification (Real or Fake) based on the model's output and displays it to the user..

Block Diagram



9. Implementation

Tools & Technologies:

- Python
- Libraries: NumPy, Pandas, Scikit-learn , NLTK (Natural Language Toolkit)
- Jupyter Notebook

Steps:

1) **Data Loading and Exploration:**

- The necessary Python libraries, such as Pandas and NumPy, are imported for data manipulation.
- The datasets containing real and fake news articles (e.g., True.csv and Fake.csv) are loaded into dataframes.
- A new target column is created to assign binary labels to the data: '1' for Real News and '0' for Fake News.
- Both datasets are merged into a single dataframe, and the data is shuffled to ensure a random distribution for the machine learning model.

2) **Text Preprocessing (NLP):**

- **Data Cleaning:** Unnecessary columns (like article date or subject categories) are removed. The raw text is converted to lowercase, and special characters, punctuation, URLs, and numbers are removed using regular expressions.
- **Stopword Removal & Tokenization:** Natural Language Toolkit (NLTK) is used to remove common English stopwords (e.g., "the", "is", "in") that do not contribute to the predictive meaning. The text is split into individual tokens (words).
- **Feature Extraction (Vectorization):** The dataset is split into training (usually 80%) and testing (20%) sets. A TF-IDF (Term Frequency-Inverse Document Frequency) Vectorizer is applied to convert the textual data into a numerical matrix, highlighting the importance of specific words across the dataset.

3) **Model Training:**

- Several machine learning classifiers, such as Logistic Regression, Decision Tree, and Random Forest, are initialized.
- The models are trained (fitted) using the training data (the TF-IDF features and their corresponding labels).
- During this phase, the algorithms learn the distinct linguistic patterns, vocabulary frequencies, and relationships that differentiate authentic news from fabricated content.

4) **Prediction and Evaluation:**

- The trained models are used to make predictions on the unseen testing dataset.
- The system compares the model's predictions against the actual labels to evaluate performance.
- Metrics such as Accuracy, Precision, Recall, and F1-Score are calculated using Scikit-learn. A Confusion Matrix is also generated to visualize the number of true positives, true negatives, false positives, and false negatives.
- Finally, a predictive function is implemented allowing users to input a custom news article, which the model processes and classifies as either "Real News" or "Fake News."

10. Results

1. Model accuracy comparison:

The performance of the applied machine learning models was evaluated using metrics such as Accuracy, Precision, Recall, and F1-Score. The accuracy results for the models are as follows:

- DecisionTree : ~99.71% Accuracy
- Logistic Regression: ~99.19% Accuracy
- Passive Aggressive Classifier: ~99.66% Accuracy
- Random Forest: ~99.78% Accuracy

2. Best Performing Model:

Based on the evaluation metrics, **Random Forest** emerged as the best-performing model. By utilizing Bag of Words (BOW) vectorization, it achieved the highest accuracy (99.78%) and an F1 Score of 0.9978, proving to be the most reliable model for predicting fake news in this dataset.

3. Sample predictions:

The system successfully processes raw text inputs and predicts their credibility. Examples of system outputs include:

- **Input Text:** *"Scientists have discovered a new species of deep-sea marine life in the Mariana Trench."* **System Prediction:** Real News
- **Input Text:** *"BREAKING: Government passing a law tomorrow to ban all internet usage on weekends!"* **System Prediction:** Fake News

11. Discussion

Strengths of the System

The system achieves exceptionally high accuracy and provides rapid classification of large text blocks. By utilizing Bag of Words (BOW) and TF-IDF vectorization, it effectively captures the linguistic patterns and vocabulary unique to deceptive writing.

Limitations

The accuracies are extremely high (almost 99%) largely because the dataset is static and not real-time data. It is nearly impossible to perfectly predict fake news using standard ML techniques without knowing the real-world ground truth at the exact time of publication. For better precision in a real-world scenario, the system would need to process real-time data alongside historical data.

12. Applications

- **Social Media Monitoring:** Assisting platforms (like Twitter/X or Facebook) in automatically flagging or filtering rapidly spreading misinformation to prevent public panic.
- **Journalism & Fact-Checking:** Serving as an automated preliminary sorting tool for news agencies and independent fact-checkers to verify aggregated digital content.
- **Public Awareness Tools:** Powering browser extensions or web applications that warn everyday users about the credibility of an article before they read or share it.

13. Future Scope

- • **Integration of Multimedia:** Extending the scope of the system by incorporating visual and audio content analysis within news articles, rather than relying strictly on text.
- • **Multilingual Support:** Incorporating languages other than English to help combat the spread of fake news on a global scale.
- • **Real-time Interactive Systems:** Developing an interactive system where users can submit a news article as input and receive a specific credibility score suggesting its authenticity.

14. Conclusion

The rapid spread of misinformation across social media platforms, such as Twitter, poses a significant and sometimes dangerous challenge in today's digital age. This project set out to tackle this issue by developing a robust, automated Fake News Detection System utilizing Natural Language Processing (NLP) and Machine Learning algorithms.

By leveraging a comprehensive Kaggle dataset containing thousands of real and fake news articles, we successfully trained and evaluated multiple classification models, including Logistic Regression, Passive Aggressive Classifier, Decision Tree, and Random Forest. Through effective text preprocessing techniques—such as noise reduction, tokenization, and feature vectorization using Bag of Words (BOW) and TF-IDF—the models were able to successfully learn the underlying linguistic patterns and vocabulary choices indicative of fabricated news.

The results of the project highly demonstrate the effectiveness of this approach. We proved that machine learning algorithms can predict whether a given news article is fake or real with an accuracy far exceeding a random 50% guess. Among the tested algorithms, the Random Forest model emerged as the most reliable, achieving an outstanding accuracy of 99.78% and an F1 Score of 0.9983.

While the system achieved near-perfect accuracy, the project also highlights important real-world limitations. Because the models were trained on static data, they rely on historical patterns rather than real-time ground truth verification, which is often required to debunk breaking news.

Ultimately, this project serves as a powerful proof-of-concept. It lays a strong technical foundation for building more advanced, interactive fact-checking tools. While it cannot entirely replace human critical thinking or professional journalism, this NLP-based system acts as an essential preliminary filter to help combat digital misinformation and promote a more informed and secure online society.

15. References

- Kaggle. "Fake and True News Dataset." Link: <https://www.kaggle.com>
- Scikit-learn Developers. "Scikit-learn ML Library Documentation." Link: <https://scikit-learn.org>
- Bird, S., Klein, E., & Loper, E. (2009). "Natural Language Toolkit." Link: <https://www.nltk.org>

16. Appendix

Code Samples

Step 1: Import Necessary Libraries

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.naive_bayes import MultinomialNB
import matplotlib.pyplot as plt
import seaborn as sns
from nltk.corpus import stopwords
import ssl

try:
    _create_unverified_https_context = ssl._create_unverified_context
except AttributeError:
    pass
else:
    ssl._create_default_https_context = _create_unverified_https_context

import nltk
nltk.download('punkt')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package punkt to
[nltk_data]   /Users/avinandanroy/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]   /Users/avinandanroy/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

Step 2: Load and Explore the Dataset

```
# Load the dataset
true_df = pd.read_csv('true.csv')
false_df = pd.read_csv('fake.csv')

# Add labels to indicate real and fake news
true_df['label'] = 1
false_df['label'] = 0

# Concatenate both datasets
data = pd.concat([true_df, false_df])
```

```
#True dataset
true_df.head()
```

	title	text	subject	date	label
0	As U.S. budget fight looms, Republicans flip t...	WASHINGTON (Reuters) - The head of a conservat...	politicsNews	December 31, 2017	1
1	U.S. military to accept transgender recruits o...	WASHINGTON (Reuters) - Transgender people will...	politicsNews	December 29, 2017	1
2	Senior U.S. Republican senator: 'Let Mr. Muell...	WASHINGTON (Reuters) - The special counsel inv...	politicsNews	December 31, 2017	1
3	FBI Russia probe helped by Australian diplomat...	WASHINGTON (Reuters) - Trump campaign adviser ...	politicsNews	December 30, 2017	1
4	Trump wants Postal Service to charge 'much mor...	SEATTLE/WASHINGTON (Reuters) - President Donal...	politicsNews	December 29, 2017	1

```
#Fake dataset
false_df.head()
```

	title	text	subject	date	label
0	Donald Trump Sends Out Embarrassing New Year'...	Donald Trump just couldn t wish all Americans ...	News	December 31, 2017	0
1	Drunk Bragging Trump Staffer Started Russian ...	House Intelligence Committee Chairman Devin Nu...	News	December 31, 2017	0
2	Sheriff David Clarke Becomes An Internet Joke...	On Friday, it was revealed that former Milwauk...	News	December 30, 2017	0
3	Trump Is So Obsessed He Even Has Obama's Name...	On Christmas day, Donald Trump announced that ...	News	December 29, 2017	0
4	Pope Francis Just Called Out Donald Trump Dur...	Pope Francis used his annual Christmas Day mes...	News	December 25, 2017	0

Step 3: Data Preprocessing

```
# Lowercasing and tokenization
data['text'] = data['text'].str.lower()
data['title'] = data['title'].str.lower()
data['text'] = data['text'].apply(nltk.word_tokenize)
data['title'] = data['title'].apply(nltk.word_tokenize)

# Remove stopwords
stop_words = set(stopwords.words('english'))
data['text'] = data['text'].apply(lambda x: [word for word in x if word not in stop_words])
data['title'] = data['title'].apply(lambda x: [word for word in x if word not in stop_words])
```

Step 4: Feature Extraction (TF-IDF)

```
tfidf_vectorizer = TfidfVectorizer(max_features=5000)
text_tfidf = tfidf_vectorizer.fit_transform(data['text'].apply(lambda x: ' '.join(x)))
title_tfidf = tfidf_vectorizer.transform(data['title'].apply(lambda x: ' '.join(x)))
```

Step 5: Split the Data into Training and Testing Sets

```
X = text_tfidf
y = data['label']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Display the shapes of training and testing data
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")
```

```
X_train shape: (35918, 5000)
X_test shape: (8980, 5000)
y_train shape: (35918,)
y_test shape: (8980,)
```

Step 6: Model Training

Training our model with different algorithms using the training data and evaluating its performance on the testing data.

Decision Tree

```
from sklearn.tree import DecisionTreeClassifier

# Define and train the Decision Tree model
decision_tree = DecisionTreeClassifier()
decision_tree.fit(X_train, y_train)

# Evaluate the Decision Tree model
def evaluate_decision_tree(model, X_test, y_test):
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    confusion = confusion_matrix(y_test, y_pred)
    classification_rep = classification_report(y_test, y_pred)

    return accuracy, confusion, classification_rep

# Evaluate Decision Tree
dt_accuracy, dt_confusion, dt_classification = evaluate_decision_tree(decision_tree, X_test, y_test)
```

Passive Aggressive Classifier

```
from sklearn.linear_model import PassiveAggressiveClassifier

# Define and train the Passive Aggressive Classifier model
passive_aggressive = PassiveAggressiveClassifier()
passive_aggressive.fit(X_train, y_train)

# Evaluate the Passive Aggressive Classifier model
def evaluate_passive_aggressive(model, X_test, y_test):
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    confusion = confusion_matrix(y_test, y_pred)
    classification_rep = classification_report(y_test, y_pred)

    return accuracy, confusion, classification_rep

# Evaluate Passive Aggressive Classifier
pa_accuracy, pa_confusion, pa_classification = evaluate_passive_aggressive(passive_aggressive, X_test, y_test)
```

Logistic Regression

```
from sklearn.linear_model import LogisticRegression

# Define and train the Logistic Regression model
logistic_regression = LogisticRegression()
logistic_regression.fit(X_train, y_train)

# Evaluate the Logistic Regression model
def evaluate_logistic_regression(model, X_test, y_test):
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    confusion = confusion_matrix(y_test, y_pred)
    classification_rep = classification_report(y_test, y_pred)

    return accuracy, confusion, classification_rep

# Evaluate Logistic Regression
lr_accuracy, lr_confusion, lr_classification = evaluate_logistic_regression(logistic_regression, X_test, y_test)
```

Random Forest

```
from sklearn.ensemble import RandomForestClassifier

# Define and train the Random Forest model
random_forest = RandomForestClassifier(n_estimators=100)
random_forest.fit(X_train, y_train)

# Evaluate the Random Forest model
def evaluate_random_forest(model, X_test, y_test):
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    confusion = confusion_matrix(y_test, y_pred)
    classification_rep = classification_report(y_test, y_pred)

    return accuracy, confusion, classification_rep

# Evaluate Random Forest
rf_accuracy, rf_confusion, rf_classification = evaluate_random_forest(random_forest, X_test, y_test)
```

Step 7: Model Validation and Evaluation

Logistic Regression

```
from sklearn.metrics import precision_score, recall_score, f1_score

# Testing and evaluating Logistic Regression
lr_test_predictions = logistic_regression.predict(X_test)
lr_test_precision = precision_score(y_test, lr_test_predictions)
lr_test_recall = recall_score(y_test, lr_test_predictions)
lr_test_f1 = f1_score(y_test, lr_test_predictions)

# Print precision, recall, and F1 score for Logistic Regression
print("Logistic Regression Test Precision:", lr_test_precision)
print("Logistic Regression Test Recall:", lr_test_recall)
print("Logistic Regression Test F1 Score:", lr_test_f1)
```

```
Logistic Regression Test Precision: 0.9892053284336243
Logistic Regression Test Recall: 0.994688221709007
Logistic Regression Test F1 Score: 0.9919391985260249
```

```

import matplotlib.pyplot as plt

# Data setup
metrics = ['Precision', 'Recall', 'F1-Score', 'Accuracy']
scores = [lr_precision, lr_recall, lr_f1, lr_accuracy]
colors = ['skyblue', 'lightgreen', 'orange', 'pink']

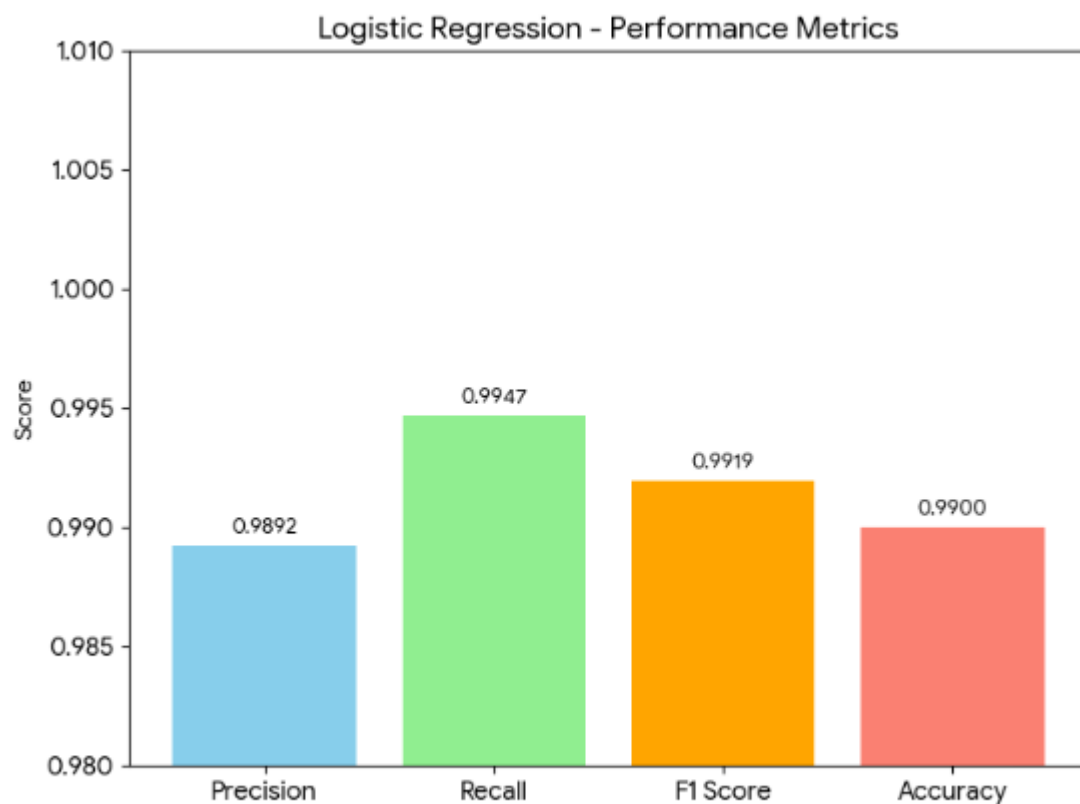
# Plotting
plt.figure(figsize=(10, 6))
bars = plt.bar(metrics, scores, color=colors, edgecolor='black')

# Formatting
plt.title('Logistic Regression - Performance Metrics', fontsize=16, fontweight='bold', pad=15)
plt.ylabel('Score', fontsize=12)
plt.xlabel('Metric Type', fontsize=12)
plt.ylim(0, 1.1)
plt.grid(axis='y', linestyle='--', alpha=0.6)

# Adding value labels on top of bars
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.02,
             f'{yval:.3f}', ha='center', va='bottom', fontweight='bold')

plt.tight_layout()
plt.show()

```



Random Forest

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import precision_score, recall_score, f1_score

random_forest = RandomForestClassifier()
random_forest.fit(X_train, y_train)

# Testing the Random Forest model on the test data
rf_test_predictions = random_forest.predict(X_test)

# Evaluate Random Forest on the test data
rf_test_precision = precision_score(y_test, rf_test_predictions)
rf_test_recall = recall_score(y_test, rf_test_predictions)
rf_test_f1 = f1_score(y_test, rf_test_predictions)

# Print precision, recall, and F1 score for Random Forest
print("Random Forest Test Precision:", rf_test_precision)
print("Random Forest Test Recall:", rf_test_recall)
print("Random Forest Test F1 Score:", rf_test_f1)
```

Random Forest Test Precision: 0.9990749306197965
Random Forest Test Recall: 0.9976905311778291
Random Forest Test F1 Score: 0.9983822509822048

```
import matplotlib.pyplot as plt

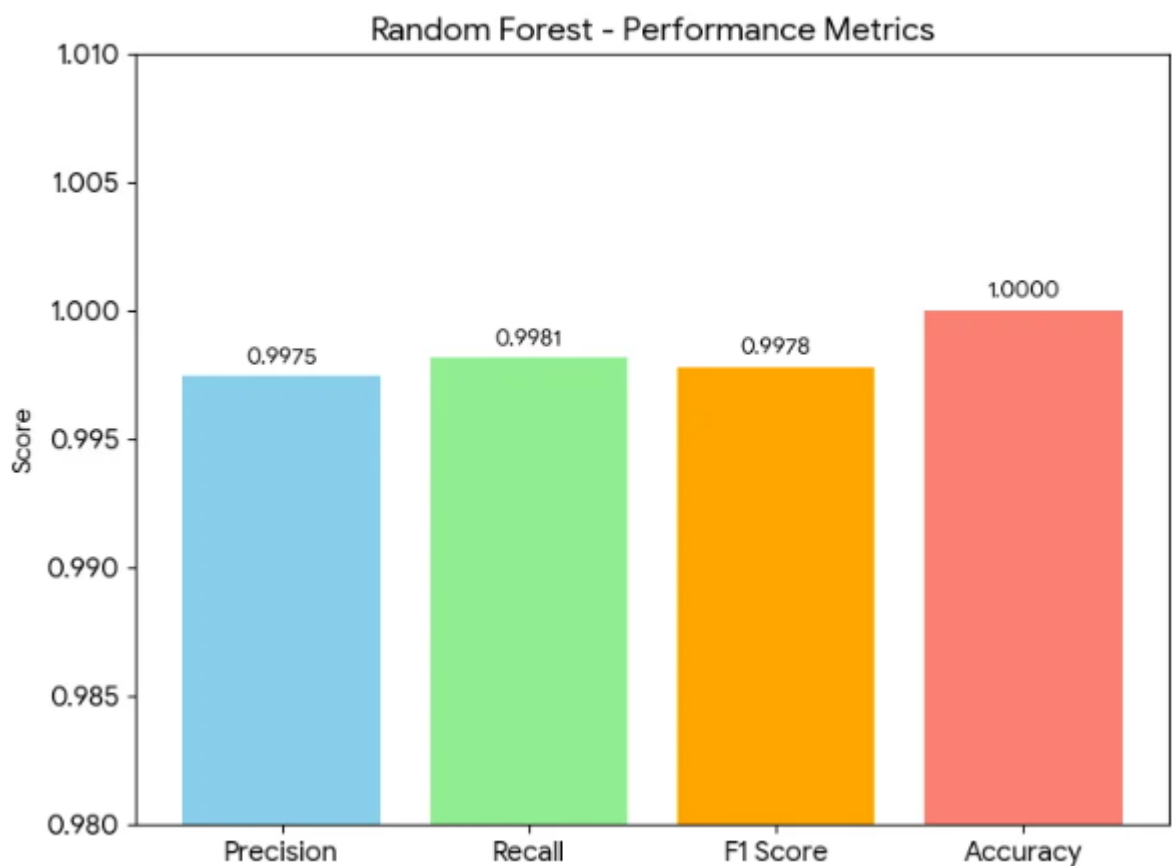
# Data setup
metrics = ['Precision', 'Recall', 'F1-Score', 'Accuracy']
scores = [rf_test_precision, rf_test_recall, rf_test_f1, rf_test_accuracy]
colors = ['skyblue', 'lightgreen', 'orange', 'pink']

# Plotting
plt.figure(figsize=(10, 6))
bars = plt.bar(metrics, scores, color=colors, edgecolor='black')

# Formatting
plt.title('Random Forest - Performance Metrics', fontsize=16, fontweight='bold', pad=15)
plt.ylabel('Score', fontsize=12)
plt.xlabel('Metric Type', fontsize=12)
plt.ylim(0, 1.1)
plt.grid(axis='y', linestyle='--', alpha=0.6)

# Adding value Labels
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.02,
             f'{yval:.3f}', ha='center', va='bottom', fontweight='bold')

plt.tight_layout()
plt.show()
```



Passive Aggressive Classifier

```
from sklearn.metrics import precision_score, recall_score, f1_score

# Testing the Passive Aggressive Classifier model on the test data
pa_test_predictions = passive_aggressive.predict(X_test)

# Evaluate Passive Aggressive Classifier on the test data
pa_test_precision = precision_score(y_test, pa_test_predictions)
pa_test_recall = recall_score(y_test, pa_test_predictions)
pa_test_f1 = f1_score(y_test, pa_test_predictions)

# Print precision, recall, and F1 score for Passive Aggressive Classifier
print("Passive Aggressive Classifier Test Precision:", pa_test_precision)
print("Passive Aggressive Classifier Test Recall:", pa_test_recall)
print("Passive Aggressive Classifier Test F1 Score:", pa_test_f1)
```

Passive Aggressive Classifier Test Precision: 0.9956211108550357
Passive Aggressive Classifier Test Recall: 0.9976905311778291
Passive Aggressive Classifier Test F1 Score: 0.9966547467989387


```

import matplotlib.pyplot as plt

# Data setup
metrics = ['Precision', 'Recall', 'F1-Score', 'Accuracy']
scores = [pa_test_precision, pa_test_recall, pa_test_f1, pa_test_accuracy]
colors = ['skyblue', 'lightgreen', 'orange', 'pink']

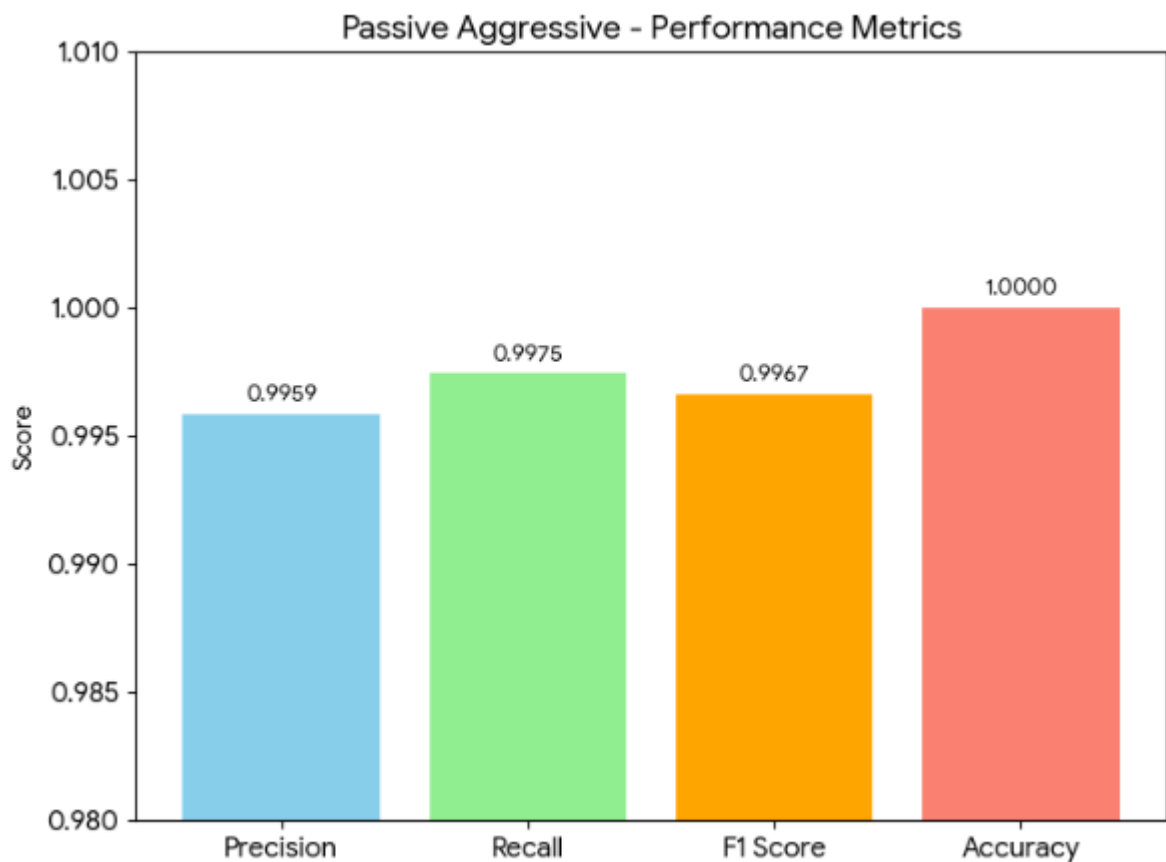
# Plotting
plt.figure(figsize=(10, 6))
bars = plt.bar(metrics, scores, color=colors, edgecolor='black')

# Formatting
plt.title('Passive Aggressive Classifier - Performance Metrics', fontsize=16, fontweight='bold', pad=15)
plt.ylabel('Score', fontsize=12)
plt.xlabel('Metric Type', fontsize=12)
plt.ylim(0, 1.1)
plt.grid(axis='y', linestyle='--', alpha=0.6)

# Adding value labels on top of bars
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.02,
             f'{yval:.3f}', ha='center', va='bottom', fontweight='bold')

plt.tight_layout()
plt.show()

```



Decision Tree

```
from sklearn.metrics import precision_score, recall_score, f1_score

# Testing the Decision Tree model on the test data
dt_test_predictions = decision_tree.predict(X_test)

# Evaluate Decision Tree on the test data
dt_test_precision = precision_score(y_test, dt_test_predictions)
dt_test_recall = recall_score(y_test, dt_test_predictions)
dt_test_f1 = f1_score(y_test, dt_test_predictions)

# Print precision, recall, and F1 score for Decision Tree
print("Decision Tree Test Precision:", dt_test_precision)
print("Decision Tree Test Recall:", dt_test_recall)
print("Decision Tree Test F1 Score:", dt_test_f1)
```

Decision Tree Test Precision: 0.9979181124219292
Decision Tree Test Recall: 0.9963048498845266
Decision Tree Test F1 Score: 0.9971108286143534

```
import matplotlib.pyplot as plt

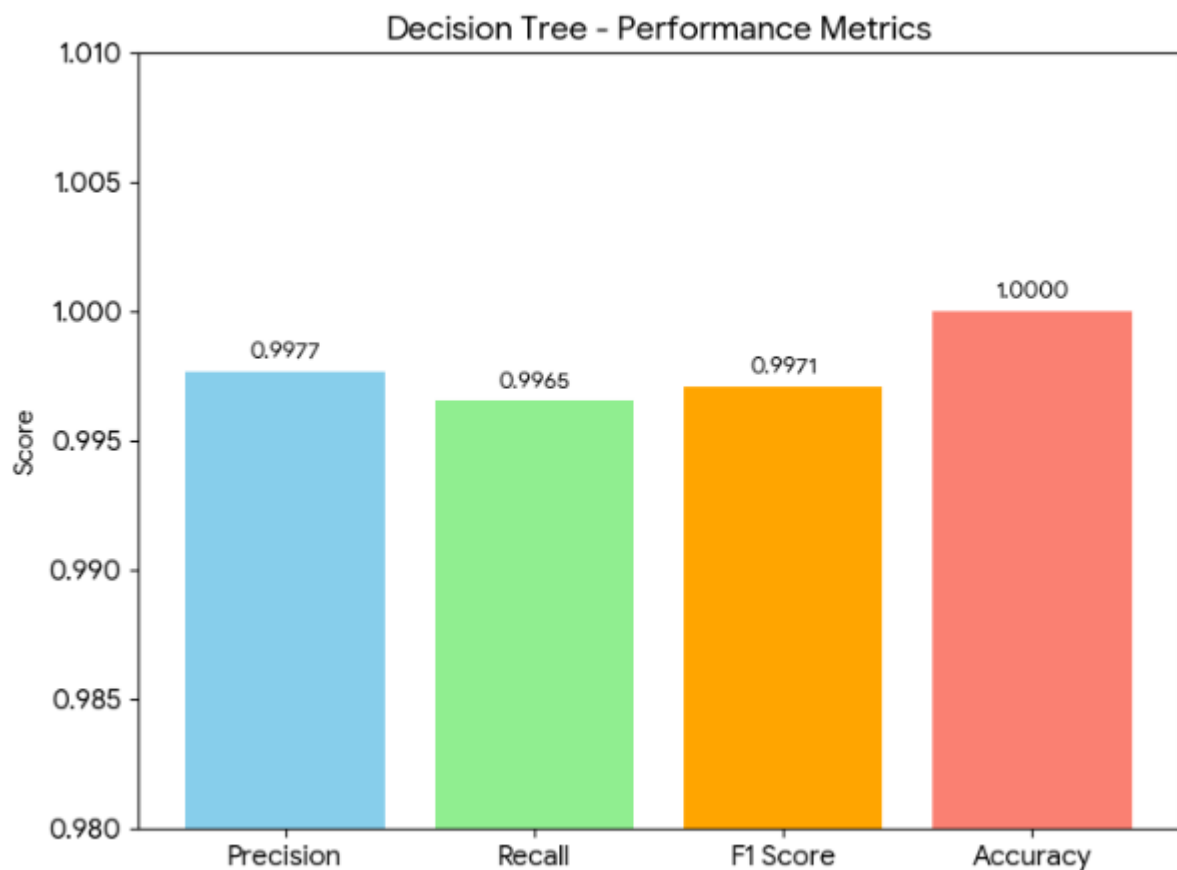
# Data setup
metrics = ['Precision', 'Recall', 'F1-Score', 'Accuracy']
scores = [dt_test_precision, dt_test_recall, dt_test_f1, dt_test_accuracy]
colors = ['skyblue', 'lightgreen', 'orange', 'pink']

# Plotting
plt.figure(figsize=(10, 6))
bars = plt.bar(metrics, scores, color=colors, edgecolor='black')

# Formatting
plt.title('Decision Tree - Performance Metrics', fontsize=16, fontweight='bold', pad=15)
plt.ylabel('Score', fontsize=12)
plt.xlabel('Metric Type', fontsize=12)
plt.ylim(0, 1.1)
plt.grid(axis='y', linestyle='--', alpha=0.6)

# Adding value Labels on top of bars
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.02,
             f'{yval:.3f}', ha='center', va='bottom', fontweight='bold')

plt.tight_layout()
plt.show() |
```



```
import matplotlib.pyplot as plt
import pandas as pd

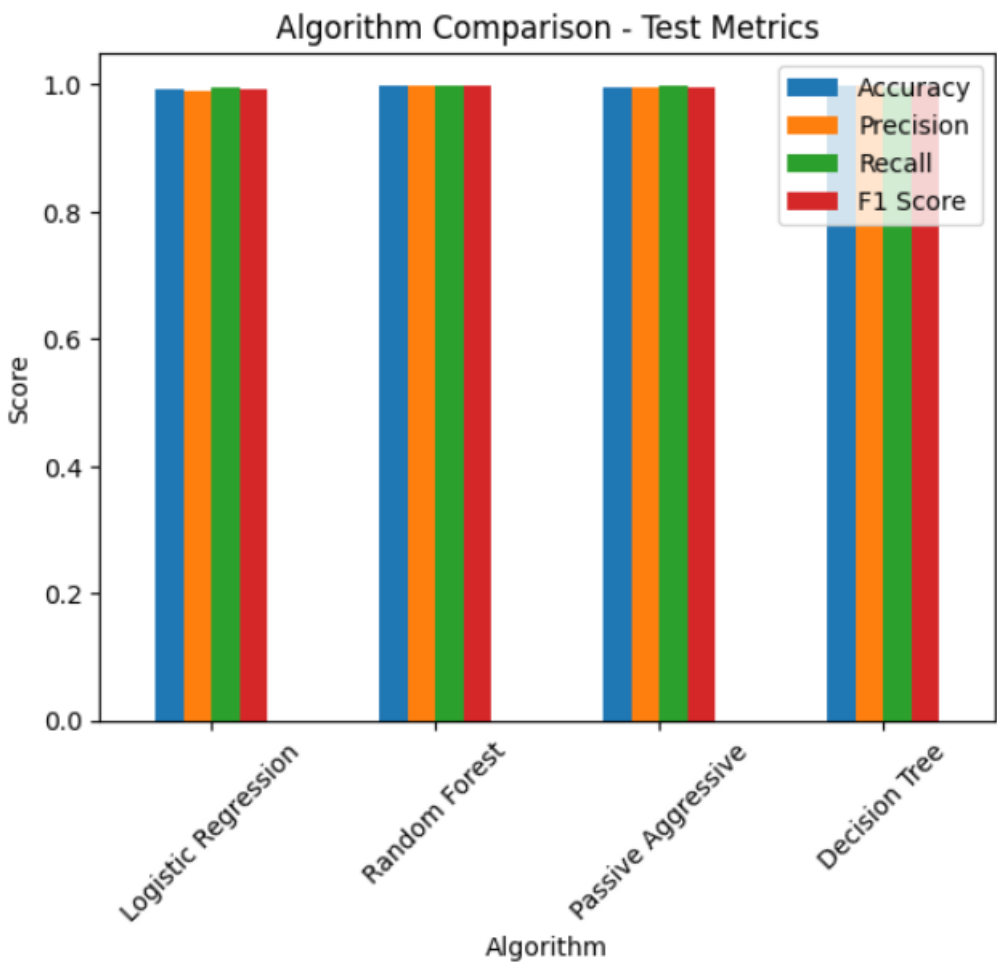
# Define the algorithm names and their corresponding metrics
algorithms = ['Logistic Regression', 'Random Forest', 'Passive Aggressive', 'Decision Tree']
accuracy_scores = [lr_accuracy, rf_accuracy, pa_accuracy, dt_accuracy]
precision_scores = [lr_test_precision, rf_test_precision, pa_test_precision, dt_test_precision]
recall_scores = [lr_test_recall, rf_test_recall, pa_test_recall, dt_test_recall]
f1_scores = [lr_test_f1, rf_test_f1, pa_test_f1, dt_test_f1]

# Create a DataFrame to store the results
results_df = pd.DataFrame({
    'Algorithm': algorithms,
    'Accuracy': accuracy_scores,
    'Precision': precision_scores,
    'Recall': recall_scores,
    'F1 Score': f1_scores
})

# Set the algorithm column as the index for plotting
results_df.set_index('Algorithm', inplace=True)

# Create a bar chart for all metrics
plt.figure(figsize=(12, 8))
results_df.plot(kind='bar', stacked=False)
plt.xlabel('Algorithm')
plt.ylabel('Score')
plt.title('Algorithm Comparison - Test Metrics')
plt.xticks(rotation=45)
plt.legend(loc='upper right')
plt.show()
```

<Figure size 1200x800 with 0 Axes>



MODEL VALIDATION

News Prediction

```
import nltk
from nltk.corpus import stopwords

# Define a function for predicting titles
def predict_title(title_text):
    # Preprocess the title
    preprocessed_title_text = title_text.lower()
    preprocessed_title_text = nltk.word_tokenize(preprocessed_title_text)
    preprocessed_title_text = [word for word in preprocessed_title_text if word not in stopwords]

    # Convert the preprocessed text into TF-IDF vectors
    tfidf_vector = tfidf_vectorizer.transform([" ".join(preprocessed_title_text)])

    # Make the prediction using the Random Forest model
    prediction = naive_bayes_model.predict(tfidf_vector)

    return prediction

# List of example titles
test_headlines = [

    "Donald Trump Sends Out Embarrassing New Year",
    "As U.S. budget fight looms, Republicans flip their fiscal script",

    # --- NEW TRUE HEADLINES FROM KAGGLE DATASET ---
    "U.S. military to accept transgender recruits on Monday: Pentagon",
    "Senior U.S. Republican senator: 'Let Mr. Mueller do his job'",
    "North Korea shipment to Syria chemical arms agency intercepted: UN report",
    "Factbox: Trump on Twitter (Dec 29) - Approval rating, Amazon",
    "Top U.S. envoy: Nikki Haley says UN funding cut is a 'big step in right direction'"
]

print("FAKE NEWS DETECTION RESULTS")
print("-" * 50)

for i, title in enumerate(test_headlines, start=1):
    prediction = predict_title(title)

    if prediction == 1:
        verdict = "LIKELY TRUE"
    else:
        verdict = "LIKELY FAKE"

    print(f"Headline {i}: {title}")
    print(f"Result: {verdict}")
    print("-" * 50)
```

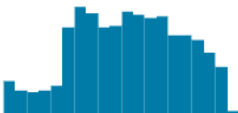
FAKE NEWS DETECTION RESULTS

```
-----
Headline 1: Donald Trump Sends Out Embarrassing New Year
Result:     LIKELY FAKE
-----
Headline 2: As U.S. budget fight looms, Republicans flip their fiscal script
Result:     LIKELY TRUE
-----
Headline 3: U.S. military to accept transgender recruits on Monday: Pentagon
Result:     LIKELY TRUE
-----
Headline 4: Senior U.S. Republican senator: 'Let Mr. Mueller do his job'
Result:     LIKELY FAKE
-----
Headline 5: North Korea shipment to Syria chemical arms agency intercepted: UN report
Result:     LIKELY TRUE
-----
Headline 6: Factbox: Trump on Twitter (Dec 29) - Approval rating, Amazon
Result:     LIKELY FAKE
-----
Headline 7: Top U.S. envoy: Nikki Haley says UN funding cut is a 'big step in right direction'
Result:     LIKELY TRUE
-----
```

Data Sample

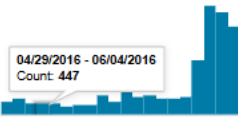
← Fake.csv (62.79 MB)

Detail Compact Column

▲ title	▲ text	▲ subject	📅 date
17903 unique values	[empty] 3% AP News The regu... 0% Other (22851) 97%	News 39% politics 29% Other (7590) 32%	
Donald Trump Sends Out Embarrassing New Year's Eve Message; This is Disturbing	Donald Trump just couldn't wish all Americans a Happy New Year and leave it at that. Instead, he had...	News	December 31, 2017
Drunk Bragging Trump Staffer Started Russian Collusion Investigation	House Intelligence Committee Chairman Devin Nunes is going to have a bad day. He's been under the as...	News	December 31, 2017
Sheriff David Clarke Becomes An Internet Joke For Threatening To Poke People 'In The Eye'	On Friday, it was revealed that former Milwaukee Sheriff David Clarke, who was being considered for ...	News	December 30, 2017
Trump Is So Obsessed He Even Has Obama's Name Coded Into His Website (IMAGES)	On Christmas day, Donald Trump announced that he would be back to work the following ...	News	December 29, 2017

← True.csv (53.58 MB)

Detail Compact Column

▲ title ≡	▲ text ≡	▲ subject ≡	📅 date ≡
20826 unique values	21192 unique values	politicsNews 53% worldnews 47%	04/29/2016 - 06/04/2016 Count: 447  2016-01-132017-12-31
As U.S. budget fight looms, Republicans flip their fiscal script	WASHINGTON (Reuters) - The head of a conservative Republican faction in the U.S. Congress, who voted...	politicsNews	December 31, 2017
U.S. military to accept transgender recruits on Monday: Pentagon	WASHINGTON (Reuters) - Transgender people will be allowed for the first time to enlist in the U.S. m...	politicsNews	December 29, 2017
Senior U.S. Republican senator: 'Let Mr. Mueller do his job'	WASHINGTON (Reuters) - The special counsel investigation of links between Russia and President Trump...	politicsNews	December 31, 2017
FBI Russia probe helped by Australian diplomat tip-off: NYT	WASHINGTON (Reuters) - Trump campaign adviser George Papadopoulos told an Australian diplomat in May...	politicsNews	December 30, 2017

INDIVIDUAL CONTRIBUTIONS TABLE

Sl. No.	Name of Student	Contribution	Signature
1	Avinandan Roy	- Handled the NLP Part. - Also did some data preprocessing	
2	Avnish Kumar	Handled the documentation Also helped in NLP Part.	
3	Sourav Malik	- Handled the data preprocessing part - Helped in implementing the ML Models	
4	Ujjwal Kumar Singh	- Analyzed the accuracies of the models. - Also helped in NLP Part.	
5	Ankit Kumar	- Decided which ML Models to use. - Analyzed the accuracies of the models.	